

Project Report: Academia – Course Registration Portal

1. Problem Statement

This project is developed as part of the **CS-513 System Software Mini Project** at IIT Bangalore.

Title: Design and Development of Course Registration Portal (Academia)

Description

The goal is to build a user-friendly and multifunctional academia portal with the following features:

- **File-based backend:** All Student, Faculty, and Course data must be stored in files.
- **User login system:** All users (Admin, Student, Faculty) log in using credentials. Admin manages user accounts.
- **Role-based access:** Users see menu options based on their role.
- **Password protection:** Ensures only authorized access to administrative operations.
- **Concurrency:** The server handles multiple clients concurrently using threads.
- **File locking:** Implements read and write locks for safe course registration and viewing.
- **System calls:** Emphasizes using system calls over library functions for file handling, threading, and IPC.

2. Implementation Details

Architecture

- **Client-Server Model:** Built using TCP sockets in C.
- **Server:** Authenticates users and provides role-specific access to academic data.
- **Client:** Connects to the server and displays an interactive menu.

User Roles and Menus

Admin

- Add Student
- Add Faculty
- Activate/Deactivate Student
- Update Student/Faculty Details
- Exit

Faculty

- Add Course
- Remove Offered Course
- View Enrollments
- Change Password
- Exit

Student

- Enroll to New Courses
- Unenroll from Enrolled Courses
- View Enrolled Courses
- Change Password
- Exit

File Handling

- Data is stored in:
 - `users.dat`
 - `students.dat`
 - `faculty.dat`
 - `courses.dat`
 - `enrollments_<course_id>.dat`
- Admin is initialized in `users.dat` with default credentials (`admin`, `admin123`).

Locking Mechanism

- **Read lock:** When viewing course details.
- **Write lock:** When enrolling or unenrolling from courses.
- Implemented using `fcntl()` file locking (`flock` and `struct flock`).

Concurrency

- Each client connection is handled in a separate thread (`pthread_create`).
- Shared file resources are protected using `pthread_mutex_t`.

3. Key Function Prototypes and Descriptions

Server Side

```
void handle_client(int client_socket);
```

Manages the complete lifecycle of a client connection.

```
int authenticate_user(char *username, char *password, User *user);
```

Validates credentials from `users.dat`.

```
void admin_menu(int client_socket, User *user);  
void faculty_menu(int client_socket, User *user);  
void student_menu(int client_socket, User *user);
```

Displays and handles role-specific menu options.

```
void add_student(int client_socket);  
void add_faculty(int client_socket);  
void activate_deactivate_student(int client_socket);  
void update_details(int client_socket);
```

Admin operations to manage users.

```
void add_course(int client_socket, User *user);  
void remove_course(int client_socket, User *user);  
void view_enrollments(int client_socket, User *user);
```

Faculty operations to manage courses.

```
void enroll_course(int client_socket, User *user);  
void unenroll_course(int client_socket, User *user);  
void view_enrolled_courses(int client_socket, User *user);
```

Student operations for course registration.

```
void send_message(int client_socket, const char *message);  
void receive_message(int client_socket, char *buffer);
```

Communication functions for sending and receiving data over sockets.

Client Side

```
void send_message(int socket_fd, const char *message);  
void receive_message(int socket_fd, char *buffer);
```

Handles client-server interaction.

```
int main();
```

Creates a socket, connects to the server, and enters the command loop.

4. Output Screenshots

```
[krishna@archlinux mini-proj]$ ./client
Connected to server

=====
                Welcome to Academia Portal
=====

+-----+
| 1. Login          |
| 2. Create Account |
+-----+
Enter your choice: 2

=====
                CREATE NEW ACCOUNT
=====

Username: admin1
Password: admin
Role (1 for Admin, 2 for Faculty, 3 for Student): 1
Account created successfully. Please login with your new credentials.
Server disconnected
```

```
=====
                ADMIN MENU
=====

+-----+
| 1. Add Student          |
| 2. Add Faculty          |
| 3. Activate/Deactivate Student |
| 4. Update Student/Faculty details |
| 5. Exit                 |
+-----+
Enter your choice:
```

```
=====
                FACULTY MENU
=====

+-----+
| 1. Add new Course          |
| 2. Remove offered Course   |
| 3. View enrollments in Courses |
| 4. Password Change         |
| 5. Exit                    |
+-----+
Enter your choice: █
```

```
=====
                        STUDENT MENU
=====

+-----+
| 1. Enroll to new Courses      |
| 2. Unenroll from already enrolled Courses |
| 3. View enrolled Courses     |
| 4. Password Change           |
| 5. Exit                      |
+-----+
Enter your choice: █
```

5. Conclusion

This project demonstrates a system-level implementation of a multi-role academic portal using:

- Socket programming
- System calls for file and thread handling
- File locking to manage concurrency
- Menu-driven role-specific access

It serves as a solid foundation for building more advanced educational tools in future coursework or production environments.